

# ARM PrimeCell Driver

SSMC v1.0 (PL093)  
API Specification

---

## PrimeCell driver API description

© Copyright ARM Limited 2001. All rights reserved.

## Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

## Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

## Product status

The information in this document is Final (information on a developed product).

## Web address

<http://www.arm.com>

---

## Feedback

ARM limited welcomes feedback on both the product, and the documentation.

### Feedback on this document

If you have any comments on about this document, please send email to [errata@arm.com](mailto:errata@arm.com) giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

### Document Summary

|                       |       |
|-----------------------|-------|
| Document Issue        | A     |
| PrimeCell Identifier  | SSMC  |
| Code Version          | 1.0   |
| PrimeCell Identifiers | PL093 |

---

# Contents

|            |                                           |           |
|------------|-------------------------------------------|-----------|
| <b>1</b>   | <b>ABOUT THIS DOCUMENT</b>                | <b>6</b>  |
| <b>1.1</b> | <b>Change control</b>                     | <b>6</b>  |
| 1.1.1      | Change history                            | 6         |
| <b>2</b>   | <b>SUMMARY</b>                            | <b>7</b>  |
| <b>3</b>   | <b>FEATURES</b>                           | <b>8</b>  |
| <b>4</b>   | <b>DRIVER DESCRIPTION</b>                 | <b>9</b>  |
| 4.1        | Overview                                  | 9         |
| 4.2        | Driver Configuration                      | 9         |
| 4.3        | Driver Initialisation                     | 9         |
| 4.4        | Setting Control Data                      | 9         |
| 4.5        | Reading Control Data                      | 9         |
| 4.6        | Configuration of the Memory Banks         | 9         |
| 4.7        | Reading Configuration of the Memory Banks | 9         |
| 4.8        | Write Protection of Memory Banks          | 9         |
| 4.9        | Reading Error Status Flag                 | 10        |
| 4.10       | Reading Status                            | 10        |
| <b>5</b>   | <b>DETAILS</b>                            | <b>11</b> |
| <b>6</b>   | <b>RELEASED FILES</b>                     | <b>11</b> |
| <b>7</b>   | <b>PUBLIC API</b>                         | <b>12</b> |
| <b>7.1</b> | <b>Type Definitions</b>                   | <b>12</b> |
| 7.1.1      | apPL093_eRBLE                             | 12        |
| 7.1.2      | apPL093_eWaitPol                          | 12        |
| 7.1.3      | apPL093_eWaitEn                           | 12        |
| 7.1.4      | apPL093_eWP                               | 13        |
| 7.1.5      | apPL093_eMW                               | 13        |
| 7.1.6      | apPL093_eSMBLSPol                         | 13        |
| 7.1.7      | apPL093_eBMRead                           | 13        |
| 7.1.8      | apPL093_eSyncReadDev                      | 14        |

---

|            |                           |           |
|------------|---------------------------|-----------|
| 7.1.9      | apPL093_eBurstLenRead     | 14        |
| 7.1.10     | apPL093_eAddrValidReadEn  | 14        |
| 7.1.11     | apPL093_eBIReadEn         | 14        |
| 7.1.12     | apPL093_eWrapReadEn       | 15        |
| 7.1.13     | apPL093_eBMWrite          | 15        |
| 7.1.14     | apPL093_eSyncWriteDev     | 15        |
| 7.1.15     | apPL093_eBurstLenWrite    | 15        |
| 7.1.16     | apPL093_eAddrValidWriteEn | 16        |
| 7.1.17     | apPL093_eBIWriteEn        | 16        |
| 7.1.18     | apPL093_eSMClockEn        | 16        |
| 7.1.19     | apPL093_eMemClkRatio      | 16        |
| 7.1.20     | apPL093_sControl          | 17        |
| 7.1.21     | apPL093_sData             | 17        |
| 7.1.22     | apPL093_eError            | 19        |
| <b>7.2</b> | <b>External Functions</b> | <b>20</b> |
| 7.2.1      | apPL093_Initialize        | 20        |
| 7.2.2      | apPL093_InitGet           | 20        |
| 7.2.3      | apPL093_StateSizeGet      | 21        |
| 7.2.4      | apPL093_MemBankConfigSet  | 21        |
| 7.2.5      | apPL093_MemBankConfigGet  | 22        |
| 7.2.6      | apPL093_WriteEnable       | 22        |
| 7.2.7      | apPL093_WriteDisable      | 23        |
| 7.2.8      | apPL093_ErrorGet          | 23        |
| 7.2.9      | apPL093_ControlSet        | 24        |
| 7.2.10     | apPL093_ControlGet        | 24        |
| 7.2.11     | apPL093_StatusGet         | 24        |

---

## 1 ABOUT THIS DOCUMENT

### 1.1 Change control

#### 1.1.1 Change history

| Issue | Date            | Change             |
|-------|-----------------|--------------------|
| A     | 14th March 2002 | First issue of API |

---

## 2 SUMMARY

The PrimeCell Synchronous Static Memory Controller (SSMC) is an Advanced Microcontroller Bus Architecture (AMBA) compliant System-on-Chip peripheral that is developed, tested and licensed by ARM.

The PrimeCell SSMC is an AMBA slave module and connects to the Advanced High-performance Bus (AHB).

The PrimeCell SSMC supports:

- asynchronous static memory-mapped devices including RAM, ROM, and flash
- synchronous static memory-mapped devices including synchronous burst flash
- asynchronous page mode read operation in non-clocked memory subsystems
- asynchronous burst mode read access from burst mode ROM and flash devices
- 8, 16, and 32-bit wide external memory data paths
- little-endian and big-endian memory architectures
- AHB burst transfers
- independent configuration for up to eight memory banks, each up to 64Mb
- programmable wait states (up to 31)
- programmable bus turnaround cycles (up to 15)
- programmable output enable and write enable delays (up to 15)
- write enable and byte lane select outputs for use with 32, 16, or 8-bit SRAM devices
- independent byte lane control for each memory bank
- external asynchronous wait control
- configurable size at reset for boot memory bank using external control pins
- system testing using externally applied TIC vectors through built-in TIC AMBA master block
- support to interface to another memory control using EBI
- multiple memory clock frequencies available: HCLK, HCLK/2 and HCLK/3

---

## 3 FEATURES

The PrimeCell SSMC driver supports independent configuration for up to eight memory banks with the following programmable parameters:

- ◆ external memory width, 8, 16, or 32-bit
- ◆ burst mode operation
- ◆ write protection
- ◆ external wait control enable
- ◆ external wait polarity
- ◆ write wait states for static RAM devices
- ◆ read wait states for static RAM and ROM devices
- ◆ initial and subsequent burst read WAIT state for burst ROM devices
- ◆ read byte lane enable control
- ◆ bus turn-around (idle) cycles
- ◆ output enable and write enable output delays



---

## 4 DRIVER DESCRIPTION

### 4.1 Overview

The SSMC driver contains the set of functions that allow the configuration of up to eight memory banks. This includes reading the configuration, disable and enable write protection, define status of error flag for each memory bank, set up and read control data for SSMC device, and read its status.

### 4.2 Driver Configuration

The SSMC driver can be configured through the macro `apPL093_BANK7_BOOT` in system description file **applatfm.h**.

If `apPL093_BANK7_BOOT` is defined in **applatfm.h**, which indicates that the system boots from Bank 7, `apPL093_Initialize()` function will not set up default reset values for Bank 7, otherwise the default configuration parameters for this memory bank will be programmed.

### 4.3 Driver Initialisation

- Call `apPL093_Initialize()` function.

### 4.4 Setting Control Data

- Call `apPL093_ControlSet()` function to set up control data.

### 4.5 Reading Control Data

- Call `apPL093_ControlGet()` function to read control data.

### 4.6 Configuration of the Memory Banks

- Call `apPL093_MemBankConfigSet()` to configure the specified memory bank.

### 4.7 Reading Configuration of the Memory Banks

- Call `apPL093_MemBankConfigGet()` to read configuration of the specified memory bank.

### 4.8 Write Protection of Memory Banks

- Call `apPL093_WriteEnable()` to enable write operations for the specified memory bank.
- Call `apPL093_WriteDisable()` to disable write operations for the specified memory bank.

---

## 4.9 Reading Error Status Flag

- Call `apPL093_ErrorGet()` to read and clear the error status flag for the specified memory bank.

## 4.10 Reading Status

- Call `apPL093_StatusGet()` function to read status of SSMC.

---

## 5 DETAILS

|                                                             |            |
|-------------------------------------------------------------|------------|
| ◆ Peak performance obtained in tests:                       | N/A        |
| ◆ ROM size (RO code + RO data) (release build):             | 1224 Bytes |
| ◆ RAM size (ZI data) per instance (release build):          | 8 Bytes    |
| ◆ Does this driver support multiple PrimeCells?:            | YES        |
| ◆ Can these be detected at run-time?                        | NO         |
| ◆ Does this driver support both endian-nesses? <sup>1</sup> | YES        |
| ◆ Does this driver support Thumb? <sup>2</sup>              | YES        |

## 6 RELEASED FILES

|                    |                       |
|--------------------|-----------------------|
| PL093-ISPC-1000-A  | This API document     |
| PL093-TREP-1000    | Test Results document |
| apPL093/apPL093.h  | Public header file    |
| apPL093/PL093.h    | Private header file   |
| apPL093/PL093.c    | Source code           |
| apPL093/PL093tst.c | Test code             |

---

<sup>1</sup> To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

<sup>2</sup> The driver may include ARM code, but this must interwork with a Thumb build.

---

## 7 PUBLIC API

### 7.1 Type Definitions

#### 7.1.1 apPL093\_eRBLE

This specifies the options for the read byte lane enables.

Declaration

```
typedef enum apPL093_eRBLE
```

Elements

|                   |                                                                                             |
|-------------------|---------------------------------------------------------------------------------------------|
| apPL093_RBLE_HIGH | nSMBLS[3:0] all deasserted HIGH during system reads from external memory (default at reset) |
| apPL093_RBLE_LOW  | nSMBLS[3:0] all deasserted LOW during system reads from external memory                     |

#### 7.1.2 apPL093\_eWaitPol

This specifies options for polarity of the external wait input for activation.

Declaration

```
typedef enum apPL093_eWaitPol
```

Elements

|                      |                                                    |
|----------------------|----------------------------------------------------|
| apPL093_WAITPOL_LOW  | The SMWAIT signal is active LOW (default at reset) |
| apPL093_WAITPOL_HIGH | The SMWAIT signal is active HIGH                   |

#### 7.1.3 apPL093\_eWaitEn

This specifies the options for the external memory supplied wait signal.

Declaration

```
typedef enum apPL093_eWaitEn
```

Elements

|                    |                                                                       |
|--------------------|-----------------------------------------------------------------------|
| apPL093_WAITEN_NO  | SSMC is not controlled by the external wait signal (default at reset) |
| apPL093_WAITEN_YES | SSMC looks for the external wait input signal SMWAIT                  |

---

#### 7.1.4 apPL093\_eWP

This specifies options for write protection of the memory bank.

Declaration

```
typedef enum apPL093_eWP
```

Elements

|                |                                        |
|----------------|----------------------------------------|
| apPL093_WP_NO  | No write protection (default at reset) |
| apPL093_WP_YES | Device is write protected              |

#### 7.1.5 apPL093\_eMW

This specifies options for memory width.

Declaration

```
typedef enum apPL093_eMW
```

Elements

|                  |        |
|------------------|--------|
| apPL093_MW_8BIT  | 8-bit  |
| apPL093_MW_16BIT | 16-bit |
| apPL093_MW_32BIT | 32-bit |

#### 7.1.6 apPL093\_eSMBLSPol

This specifies the polarity of the signal SMBLS.

Declaration

```
typedef enum apPL093_eSMBLSPol
```

Elements

|                       |                                                   |
|-----------------------|---------------------------------------------------|
| apPL093_SMBLSPOL_LOW  | The SMBLS signal is active LOW (default at reset) |
| apPL093_SMBLSPOL_HIGH | The SMBLS signal is active HIGH                   |

#### 7.1.7 apPL093\_eBMRead

This specifies options for burst mode read and asynchronous page mode.

Declaration

```
typedef enum apPL093_eBMRead
```

Elements

|                         |                                                               |
|-------------------------|---------------------------------------------------------------|
| apPL093_NON_BURST_READS | Non-burst memory devices (default at reset)                   |
| apPL093_BURST_READS     | Burst mode / asynchronous page mode reads from memory devices |

---

### 7.1.8 apPL093\_eSyncReadDev

This specifies options for synchronous access capable device connection for reads.

Declaration

```
typedef enum apPL093_eSyncReadDev
```

Elements

|                    |                                        |
|--------------------|----------------------------------------|
| apPL093_ASYNC_READ | Asynchronous device (default at reset) |
| apPL093_SYNC_READ  | Synchronous device                     |

### 7.1.9 apPL093\_eBurstLenRead

This specifies options for burst transfer length, used to set the number of sequential transfers supported by the burst device for a read.

Declaration

```
typedef enum apPL093_eBurstLenRead
```

Elements

|                       |                                     |
|-----------------------|-------------------------------------|
| apPL093_BL_READ_4BIT  | 4-transfer burst (default)          |
| apPL093_BL_READ_8BIT  | 8-transfer burst                    |
| apPL093_BL_READ_16BIT | 16-transfer burst                   |
| apPL093_BL_READ_CONT  | Continuous burst (synchronous only) |

### 7.1.10 apPL093\_eAddrValidReadEn

This specifies options for the behaviour of the signal SMADDRVALID during read operations.

Declaration

```
typedef enum apPL093_eAddrValidReadEn
```

Elements

|                     |                                                                        |
|---------------------|------------------------------------------------------------------------|
| apPL093_AVRE_HIGH   | Signal always HIGH                                                     |
| apPL093_AVRE_ACTIVE | Signal active for asynchronous and synchronous read accesses (default) |

### 7.1.11 apPL093\_eBIReadEn

This specifies options for the behavior of the signals SMBAA and SMIND during read operations.

Declaration

```
typedef enum apPL093_eBIReadEn
```

Elements

|                       |                                                        |
|-----------------------|--------------------------------------------------------|
| apPL093_BIREAD_HIGH   | Signals always HIGH                                    |
| apPL093_BIREAD_ACTIVE | Signals active for synchronous read accesses (default) |

---

### 7.1.12 apPL093\_eWrapReadEn

This enables the wrapping burst feature from external memory.

Declaration

```
typedef enum apPL093_eWrapReadEn
```

Elements

|                           |                         |
|---------------------------|-------------------------|
| apPL093_WRAPREAD_DISABLED | Wrap disabled (default) |
| apPL093_WRAPREAD_ENABLED  | Wrap enabled            |

### 7.1.13 apPL093\_eBMWrite

This specifies options for burst mode write.

Declaration

```
typedef enum apPL093_eBMWrite
```

Elements

|                          |                                                       |
|--------------------------|-------------------------------------------------------|
| apPL093_NON_BURST_WRITES | Non-burst writes to memory devices (default at reset) |
| apPL093_BURST_WRITES     | Burst mode writes to memory devices                   |

### 7.1.14 apPL093\_eSyncWriteDev

This specifies options for synchronous access capable device connection for writes.

Declaration

```
typedef enum apPL093_eSyncWriteDev
```

Elements

|                     |                                        |
|---------------------|----------------------------------------|
| apPL093_ASYNC_WRITE | Asynchronous device (default at reset) |
| apPL093_SYNC_WRITE  | Synchronous device                     |

### 7.1.15 apPL093\_eBurstLenWrite

This specifies options for burst transfer length, used to set the number of sequential transfers supported by the burst device for a write.

Declaration

```
typedef enum apPL093_eBurstLenWrite
```

Elements

|                       |                                     |
|-----------------------|-------------------------------------|
| apPL093_BL_WRITE_4BIT | 4-transfer burst (default)          |
| apPL093_BL_WRITE_8BIT | 8-transfer burst                    |
| apPL093_BL_WRITE_CONT | Continuous burst (synchronous only) |

---

### 7.1.16 apPL093\_eAddrValidWriteEn

This specifies options for the behaviour of the signal SMADDRVALID during write operations.

Declaration

```
typedef enum apPL093_eAddrValidWriteEn
```

Elements

|                     |                                                                         |
|---------------------|-------------------------------------------------------------------------|
| apPL093_AVWE_HIGH   | Signal always HIGH                                                      |
| apPL093_AVWE_ACTIVE | Signal active for asynchronous and synchronous write accesses (default) |

### 7.1.17 apPL093\_eBIWriteEn

This specifies options for the behaviour of the signals SMBAA and SMIND during write operations.

Declaration

```
typedef enum apPL093_eBIWriteEn
```

Elements

|                        |                                                         |
|------------------------|---------------------------------------------------------|
| apPL093_BIWRITE_HIGH   | Signals always HIGH                                     |
| apPL093_BIWRITE_ACTIVE | Signals active for synchronous write accesses (default) |

### 7.1.18 apPL093\_eSMClockEn

This specifies options for SMCLK enable.

Declaration

```
typedef enum apPL093_eSMClockEn
```

Elements

|                              |                                                        |
|------------------------------|--------------------------------------------------------|
| apPL093_SMCLK_ACTIVE_ACCESS  | Clock only active during memory accesses to safe power |
| apPL093_SMCLK_ALWAYS_RUNNING | Clock always running (default)                         |

### 7.1.19 apPL093\_eMemClkRatio

This specifies options for SMMemCLK to HCLK ratio.

Declaration

```
typedef enum apPL093_eMemClkRatio
```

Elements

|                         |                           |
|-------------------------|---------------------------|
| apPL093_SMMEMCLK_HCLK   | SMMemCLK = HCLK (default) |
| apPL093_SMMEMCLK_HCLK_2 | SMMemCLK = HCLK/2         |
| apPL093_SMMEMCLK_HCLK_3 | SMMemCLK = HCLK/3         |



---

### 7.1.20 apPL093\_sControl

The data structure to hold control parameters of SSMC device.

Declaration

```
struct apPL093_sControl
{
    apPL093_eSMClockEn      eSMClockEn;
    apPL093_eMemClkRatio    eMemClkRatio;
}
```

Elements

|              |                        |
|--------------|------------------------|
| eSMClockEn   | SMCLK enable           |
| eMemClkRatio | SMMemCLK to HCLK ratio |

### 7.1.21 apPL093\_sData

The data structure to hold parameters for initialisation of each memory bank.

Declaration

```
struct apPL093_sData
{
    apPL093_eRBLE          eRBLE;
    apPL093_eWaitPol       eWaitPol;
    apPL093_eWaitEn        eWaitEn;
    apPL093_eWP            eWriteProtect;
    apPL093_eMW            eMemoryWidth;
    apPL093_eSMBLSPol      eSMBLSPol;
    apPL093_eBMRead        eBMRead;
    apPL093_eSyncReadDev   eSyncReadDev;
    apPL093_eBurstLenRead  eBurstLenRead;
    apPL093_eAddrValidReadEn eAddrValidReadEn;
    apPL093_eBIReadEn      eBIReadEn;
    apPL093_eWrapReadEn    eWrapReadEn;
    apPL093_eBMWrite       eBMWrite;
    apPL093_eSyncWriteDev  eSyncWriteDev;
    apPL093_eBurstLenWrite eBurstLenWrite;
    apPL093_eAddrValidWriteEn eAddrValidWriteEn;
    apPL093_eBIWriteEn     eBIWriteEn;
    UBYTE8                 IdleTurnAroundCycles;
    UBYTE8                 ReadAccessCycles;
    UBYTE8                 WriteAccessCycles;
    UBYTE8                 OutputEnAssertDelay;
    UBYTE8                 WriteEnAssertDelay;
    UBYTE8                 BurstReadDelay;
}
```

---

## Elements

|                      |                                                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------|
| eRBLE                | Read byte lane enable                                                                                                           |
| eWaitPol             | Polarity of the external wait signal for activation                                                                             |
| eWaitEn              | External memory controller wait signal enable                                                                                   |
| eWriteProtect        | Write protection                                                                                                                |
| eMemoryWidth         | Memory width, 8, 16, or 32-bit                                                                                                  |
| eSMBLSPol            | Polarity of the signal SMBLS                                                                                                    |
| eBMRead              | Burst mode read and asynchronous page mode                                                                                      |
| eSyncReadDev         | Synchronous access for read                                                                                                     |
| eBurstLenRead        | Burst transfer length for read                                                                                                  |
| eAddrValidReadEn     | Behavior of the signal SMADDRVALID during read operations                                                                       |
| eBIReadEn            | Behavior of the signals SMBAA and SMIND during read operations                                                                  |
| eWrapReadEn          | Enables the wrapping burst feature from external memory                                                                         |
| eBMWrite             | Burst mode write                                                                                                                |
| eSyncWriteDev        | Synchronous access for write                                                                                                    |
| eBurstLenWrite       | Burst transfer length for write                                                                                                 |
| eAddrValidWriteEn    | Behavior of the signal SMADDRVALID during write operations                                                                      |
| eBIWriteEn           | Behavior of the signals SMBAA and SMIND during write operations                                                                 |
| IdleTurnAroundCycles | Idle or turn-around clock cycles, IDCY (0–15, defaults to 15 at reset). The delay is $IDCY \times t_{SMMemCLK}$                 |
| ReadAccessCycles     | Read wait state in clock cycles, WSTRD (0–31, defaults to 31 at reset). The delay is $WSTRD \times t_{SMMemCLK}$                |
| WriteAccessCycles    | Write wait state in clock cycles, WSTWR (0–31, defaults to 31 at reset). The delay is $WSTWR \times t_{SMMemCLK}$               |
| OutputEnAssertDelay  | Output enable assertion delay in clock cycles, WSTOEN (0–15, defaults to 0 at reset). The delay is $WSTOEN \times t_{SMMemCLK}$ |
| WriteEnAssertDelay   | Write enable assertion delay in clock cycles, WSTWEN (0–15, defaults to 0 at reset). The delay is $WSTWEN \times t_{SMMemCLK}$  |
| BurstReadDelay       | Burst read wait state in clock cycles, WSTBRD (0–31, defaults to 31 at reset). The delay is $WSTBRD \times t_{SMMemCLK}$        |

---

### 7.1.22 apPL093\_eError

This specifies the general errors reported by this module.  
Errors will be returned typecast to the general error type apError.

#### Declaration

```
typedef enum apPL093_eError
```

#### Elements

|                            |                                                |
|----------------------------|------------------------------------------------|
| apERR_PL093_INVALID_BANK   | Invalid bank number                            |
| apERR_PL093_INVALID_WSTRD  | Invalid value of wait states for read access   |
| apERR_PL093_INVALID_WSTWR  | Invalid value of wait states for write access  |
| apERR_PL093_INVALID_IDCY   | Invalid value of duty or turn-around cycles    |
| apERR_PL093_INVALID_WSTOEN | Invalid value of output enable assertion delay |
| apERR_PL093_INVALID_WSTWEN | Invalid value of write enable assertion delay  |
| apERR_PL093_INVALID_WSTBRD | Invalid value of burst read delay              |

---

## 7.2 External Functions

### 7.2.1 apPL093\_Initialize

Initialises the SSMC device.

#### Declaration

```
PUBLIC void apPL093_Initialize( ap0S_PL093_oId          oId,  
                             ap0S_System_eBaseAddress  eBase,  
                             UWORD32                  Interrupts,  
                             CONST ap0S_INT_oInterruptSource * pSources,  
                             apPL093_sInitialData      * pInitial )
```

#### Parameters

|            |                                                      |
|------------|------------------------------------------------------|
| old        | Device identifier                                    |
| eBase      | Base address of SSMC device control registers        |
| Interrupts | Ignored (there are no interrupts for the controller) |
| pSources   | Not used, pass as aNULL                              |
| pInitial   | Not used, pass as aNULL                              |

#### Return Value

None

#### Remarks

This function configures all memory banks with default reset parameters and marks all banks as non-initialised by writing a zero value in to the initialisation state variable. If macro apPL093\_BANK7\_BOOT is defined in **applatfm.h**, default parameters for Bank 7 are not set.

### 7.2.2 apPL093\_InitGet

This function returns the current value of the initialisation state variable.

#### Declaration

```
PUBLIC UBYTE8 apPL093_InitGet(ap0S_PL093_oId oId )
```

#### Parameters

|     |                   |
|-----|-------------------|
| old | Device identifier |
|-----|-------------------|

#### Return Value

Initialisation flags for each memory bank

#### Remarks

This bit is set if the corresponding memory bank has been initialised.

---

### 7.2.3 apPL093\_StateSizeGet

Retrieves the size required for storage of the driver state data.

#### Declaration

```
PUBLIC UWORD32 apPL093_StateSizeGet (void)
```

#### Return Value

Size (in bytes) required

#### Remarks

This function is required if apOS\_NO\_STATIC\_STATE is defined as TRUE.

### 7.2.4 apPL093\_MemBankConfigSet

This function configures the specified memory bank.

#### Declaration

```
PUBLIC apError apPL093_MemBankConfigSet( apOS_PL093_oId      oId,  
                                         UWORD32          MemBank,  
                                         CONST apPL093_sData * pBankData )
```

#### Parameters

|           |                                                            |
|-----------|------------------------------------------------------------|
| old       | Device identifier                                          |
| MemBank   | The bank number (0–7) to be configured                     |
| pBankData | Pointer to a block of data for configuring the memory bank |

#### Return Value

|                            |                                                                                  |
|----------------------------|----------------------------------------------------------------------------------|
| apERR_NONE                 | Configuration parameters set, no errors                                          |
| apERR_PL093_INVALID_BANK   | Parameters not set because of the invalid value of bank number                   |
| apERR_PL093_INVALID_WSTRD  | Parameters not set because of the invalid value of wait states for read access   |
| apERR_PL093_INVALID_WSTWR  | Parameters not set because of the invalid value of wait states for write access  |
| apERR_PL093_INVALID_IDCY   | Parameters not set because of the invalid value of duty or turn-around cycles    |
| apERR_PL093_INVALID_WSTOEN | Parameters not set because of the invalid value of output enable assertion delay |
| apERR_PL093_INVALID_WSTWEN | Parameters not set because of the invalid value of write enable assertion delay  |
| apERR_PL093_INVALID_WSTBRD | Parameters not set because of the invalid value of burst read delay              |

#### Remarks

This function writes the initialisation values in registers for the specified memory bank and sets the corresponding bit in the initialisation state variable.

---

### 7.2.5 apPL093\_MemBankConfigGet

Reads configuration of the specified memory bank.

#### Declaration

```
PUBLIC apError apPL093_MemBankConfigGet( ap0S_PL093_oId      oId,
                                         UWORD32          MemBank,
                                         apPL093_sData * CONST pBankData )
```

#### Parameters

|           |                                                                         |
|-----------|-------------------------------------------------------------------------|
| old       | Device identifier                                                       |
| MemBank   | The bank number (0–7) to read configuration                             |
| pBankData | Pointer to a block of data for reading configuration of the memory bank |

#### Return Value

|                          |                                                                 |
|--------------------------|-----------------------------------------------------------------|
| apERR_NONE               | Configuration parameters read, no errors                        |
| apERR_PL093_INVALID_BANK | Parameters not read because of the invalid value of bank number |

#### Remarks

This function reads values of bank registers and decodes them into configuration parameters.

### 7.2.6 apPL093\_WriteEnable

Enables write operations in external memory for the specified memory bank.

#### Declaration

```
PUBLIC apError apPL093_WriteEnable( ap0S_PL093_oId oId,
                                   UWORD32          MemBank )
```

#### Parameters

|         |                                           |
|---------|-------------------------------------------|
| old     | Device identifier                         |
| MemBank | The bank number (0–7) to be write enabled |

#### Return Value

|                          |                                                                           |
|--------------------------|---------------------------------------------------------------------------|
| apERR_NONE               | Memory bank write enabled, no errors                                      |
| apERR_PL093_INVALID_BANK | Memory bank not write enabled because of the invalid value of bank number |

#### Remarks

After calling this function the specified memory bank can be written to.

---

### 7.2.7 apPL093\_WriteDisable

Disables write operations in external memory for the specified memory bank.

#### Declaration

```
PUBLIC apError apPL093_WriteDisable( ap0S_PL093_oId oId,  
                                     UWORD32      MemBank )
```

#### Parameters

|         |                                            |
|---------|--------------------------------------------|
| old     | Device identifier                          |
| MemBank | The bank number (0–7) to be write disabled |

#### Return Value

|                          |                                                                            |
|--------------------------|----------------------------------------------------------------------------|
| apERR_NONE               | Memory bank write disabled, no errors                                      |
| apERR_PL093_INVALID_BANK | Memory bank not write disabled because of the invalid value of bank number |

#### Remarks

After calling this function the specified memory bank will be write protected.

### 7.2.8 apPL093\_ErrorGet

Reads the error status flag for the specified memory bank.

#### Declaration

```
PUBLIC apError apPL093_ErrorGet( ap0S_PL093_oId oId,  
                                UWORD32      MemBank,  
                                BOOL * CONST  pWaitToutErr )
```

#### Parameters

|              |                                                                                                    |
|--------------|----------------------------------------------------------------------------------------------------|
| old          | Device identifier                                                                                  |
| MemBank      | The bank number (0–7) to read error status flag                                                    |
| pWaitToutErr | Pointer to the variable that is set TRUE if there was external wait timeout error, FALSE otherwise |

#### Return Value

|                          |                                                                        |
|--------------------------|------------------------------------------------------------------------|
| apERR_NONE               | Error status flag read, no errors                                      |
| apERR_PL093_INVALID_BANK | Error status flag not read because of the invalid value of bank number |

#### Remarks

This function reads and clears error status bit of SSMC Bank Status Register for the specified memory bank.

---

### 7.2.9 apPL093\_ControlSet

Sets control data for SSMC device.

#### Declaration

```
PUBLIC void apPL093_ControlSet( ap0S_PL093_oId      oId,  
                               CONST apPL093_sControl * pControl )
```

#### Parameters

|          |                                                                            |
|----------|----------------------------------------------------------------------------|
| old      | Device identifier                                                          |
| pControl | Pointer to apPL093_sControl structure containing control data to be set up |

#### Remarks

This function writes control data stored in apPL093\_sControl structure to SSMC Control Register.

### 7.2.10 apPL093\_ControlGet

Reads control data of SSMC device.

#### Declaration

```
PUBLIC void apPL093_ControlGet( ap0S_PL093_oId      oId,  
                               apPL093_sControl * CONST pControl )
```

#### Parameters

|          |                                                                    |
|----------|--------------------------------------------------------------------|
| old      | Device identifier                                                  |
| pControl | Pointer to apPL093_sControl structure to contain read control data |

#### Remarks

This function reads value of SSMC Control Register and puts control data in apPL093\_sControl structure.

### 7.2.11 apPL093\_StatusGet

Reads the External Wait Status flag of SSMC device.

#### Declaration

```
PUBLIC BOOL apPL093_StatusGet( ap0S_PL093_oId oId )
```

#### Parameters

|     |                   |
|-----|-------------------|
| old | Device identifier |
|-----|-------------------|

#### Return Value

TRUE if SMWAIT signal is asserted  
FALSE if SMWAIT signal is deasserted.

#### Remarks

This function reads SSMC Status Register and returns state of SMWAIT signal.